UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

UTEQ

Sistema de Gestión de Pre-Sustentaciones

Documentación Técnica — Backend & Frontend

Versiones: Backend v6 · Frontend v7

Tecnologías: Spring Boot 3.2 · Angular 17 · PostgreSQL · JWT

1. Descripción General del Sistema

El Sistema de Gestión de Pre-Sustentaciones de la UTEQ es una aplicación web fullstack diseñada para digitalizar y automatizar el proceso de pre-sustentación de trabajos de titulación. El sistema permite que los estudiantes registren sus solicitudes, carguen sus anteproyectos en PDF, reciban notificaciones, consulten su cronograma y revisen sus calificaciones finales, mientras que los administradores y docentes gestionan todo el flujo del proceso desde la aprobación hasta la generación del acta digital.

1.1 Arquitectura General

Tipo de arquitectura	Cliente-Servidor (REST API + SPA)
Backend	Spring Boot 3.2.1 / Java 17 — REST API en puerto 8080
Frontend	Angular 17 (TypeScript) — SPA en puerto 4200
Base de datos	PostgreSQL — ORM con Spring Data JPA / Hibernate
Autenticación	JWT (JSON Web Tokens) con Spring Security
Comunicación	HTTP/JSON con CORS habilitado para localhost:4200

1.2 Roles de Usuario

- ESTUDIANTE: Registra solicitudes, carga anteproyectos PDF, consulta notas y horario.
- DOCENTE / JURADO: Revisa anteproyectos, actúa como presidente, vocal 1, vocal 2 o tutor.
- ADMIN: Aprueba/rechaza solicitudes, programa cronogramas, evalúa y genera actas.

2. Backend — Spring Boot

2.1 Tecnologías y Dependencias Principales

Framework base	Spring Boot 3.2.1 (spring-boot-starter-parent)
Lenguaje	Java 17
Seguridad	Spring Security + JWT (jjwt-api 0.12.5)
Persistencia	Spring Data JPA + Hibernate + PostgreSQL Driver
Utilidades	Lombok (reduce boilerplate), Bean Validation
Generación PDF	iText / OpenPDF (para actas)
Gestión de archivos	Spring Multipart — subida de PDFs de anteproyecto
Puerto	8080 (configurable en application.properties)

2.2 Configuración (application.properties)

El archivo src/main/resources/application.properties centraliza la configuración del sistema:

- spring.datasource.url: URL de conexión a PostgreSQL. Por defecto: jdbc:postgresql://localhost:5432/presusDb
- spring.datasource.username / password: Credenciales de la BD (variables de entorno DB_USERNAME / DB_PASSWORD).
- spring.jpa.hibernate.ddl-auto=update: Hibernate actualiza el esquema automáticamente al iniciar.
- jwt.secret: Clave secreta para firmar los tokens JWT (variable de entorno JWT_SECRET).
- jwt.expiration=86400000: Los tokens duran 24 horas.
- app.upload.dir: Carpeta donde se guardan los PDF de anteproyectos (por defecto uploads/anteproyectos).
- spring.servlet.multipart.max-file-size=10MB: Tamaño máximo de archivos cargados.

2.3 Estructura de Paquetes

controllers/	Controladores REST — definen los endpoints de la API
services/	Interfaces e implementaciones de la lógica de negocio
repositories/	Interfaces JPA para acceso a la base de datos
entities/	Entidades JPA mapeadas a las tablas de PostgreSQL
security/	Configuración JWT, filtros de autenticación y Spring Security
config/	SecurityConfig (reglas de acceso) y WebConfig (CORS)

3. Modelo de Datos — Entidades JPA

Cada entidad Java se mapea a una tabla en PostgreSQL usando anotaciones JPA. Se utiliza Lombok para generar getters, setters y constructores automáticamente.

3.1 Usuario (tabla: usuarios)

Entidad central que representa a cualquier persona que usa el sistema.

id	Long — Clave primaria autoincremental
nombre / apellido	String — Nombre completo del usuario
email	String único — Usado como credencial de login
password	String — Contraseña encriptada con BCrypt
rol	String — ESTUDIANTE, DOCENTE o ADMIN
activo	Boolean — Indica si la cuenta está habilitada

3.2 Estudiante (tabla: estudiante)

Perfil extendido ligado a un Usuario con rol ESTUDIANTE.

usuario	@ManyToOne → Usuario (unique: un usuario = un estudiante)
carrera	String — Nombre de la carrera universitaria
semestre	String — Semestre actual del estudiante
telefono	String — Número de contacto
expedienteCodigo	String único — Código del expediente académico

3.3 Docente (tabla: docente)

Perfil extendido ligado a un Usuario con rol DOCENTE.

usuario	@ManyToOne → Usuario
areaEspecialidad	String — Área académica de especialización
cargaHorariaSemanal	Integer — Horas semanales asignadas
disponible	Boolean — Si puede ser asignado como jurado o tutor

3.4 Solicitud (tabla: solicitud)

Entidad principal del flujo. Cada solicitud representa un proceso de pre-sustentación.

estudiante	@ManyToOne → Estudiante

tituloTema	String — Título del trabajo de titulación
modalidad	String — Modalidad del trabajo
estado	String — PENDIENTE, ENVIADA, APROBADA, RECHAZADA
fechaRegistro	LocalDateTime — Fecha de creación (automática)
observaciones	Text — Comentarios del proceso
creadoPor / actualizadoPor	@ManyToOne → Usuario (trazabilidad)

3.5 Anteproyecto (tabla: anteproyectos)

solicitud	@OneToOne → Solicitud
archivoPdf	String — Nombre del archivo en disco (carpeta uploads)
fechaEnvio	LocalDate — Fecha de carga del PDF
estado	String — ENVIADO, APROBADO, RECHAZADO
observaciones	Text — Observaciones del revisor

3.6 Cronograma (tabla: cronograma)

solicitud	@OneToOne → Solicitud
sala	@ManyToOne → Sala
fechaInicio	LocalDateTime — Fecha y hora de la sustentación
duracionMin	Integer — Duración en minutos (default 45)
estado	String — ACTIVO
getFechaFin()	Método calculado: fechaInicio + duracionMin

3.7 Jurado (tabla: jurados)

docente	@ManyToOne → Docente
solicitud	@ManyToOne → Solicitud
rol	String — PRESIDENTE, VOCAL_1, VOCAL_2
confirmado	Boolean — Si ya confirmó su asistencia

3.8 Tutor (tabla: tutores)

docente	@ManyToOne → Docente
solicitud	@OneToOne → Solicitud (único por solicitud)

estado	String — ACTIVO, FINALIZADO, REEMPLAZADO
fechaAsignacion	LocalDateTime — Automática al persistir

3.9 Evaluacion (tabla: evaluaciones)

Implementa la ponderación 60/40 configurable.

notaInstructor	Double — Nota del docente de titulación
notaJurado	Double — Nota promedio del tribunal
pesoInstructor	Double — Porcentaje del instructor (default 60%)
pesoJurado	Double — Porcentaje del jurado (default 40%)
notaFinal	Double — Calculada: (notaInst * peso/100) + (notaJur * peso/100)
resultado	String — APROBADO (>= 7.0) o REPROBADO
calcularNotaFinal()	Método helper que recalcula y establece resultado

3.10 Acta (tabla: actas)

Registra las firmas digitales de todos los actores del proceso.

solicitud	@OneToOne → Solicitud
archivoPdf	String — Ruta del PDF del acta generado
firmadaPresidente / Vocal1 / Vocal2 / Tutor	Boolean — Estado de firma por actor
fechaFirmaXxx	LocalDateTime — Marca de tiempo de cada firma
firmada	Boolean — true solo cuando TODOS firmaron
actualizarEstadoFirma()	Actualiza 'firmada' según el estado de todos los actores
getFirmantesPendientes()	Retorna texto con los actores que faltan por firmar

3.11 Otras Entidades

Sala	Sala física: codigo, nombre, capacidad, disponible
Rubrica	Criterio de evaluación: nombre, descripcion, puntajeMaximo
Notificacion	Mensaje para usuario: mensaje (Text), fecha, leida (Boolean)

4. Seguridad — JWT y Spring Security

4.1 Flujo de Autenticación

El sistema usa autenticación sin estado (stateless) basada en JSON Web Tokens:

- Paso 1: El usuario envía su email y password al endpoint POST /api/auth/login.
- Paso 2: Spring Security verifica las credenciales contra la base de datos (CustomUserDetailsService).
- Paso 3: Si son correctas, JwtTokenProvider genera un token firmado con la clave secreta (HS256).
- Paso 4: El backend devuelve el token junto con el id, email, nombre y rol del usuario.
- Paso 5: El frontend almacena el token en localStorage y lo adjunta en cada petición siguiente dentro del header Authorization: Bearer <token>.
- Paso 6: JwtAuthenticationFilter intercepta cada request, valida el token y carga el usuario en el contexto de seguridad.

4.2 Clases de Seguridad

JwtTokenProvider	Genera y valida tokens JWT. Extrae el username (email) del token.
JwtAuthenticationFilter	Filtro que se ejecuta en cada request: lee el header Authorization, valida el token y autentica al usuario.
CustomUserDetailsService	Implementa UserDetailsService de Spring. Busca al usuario por email en la BD.
SecurityConfig	Define qué rutas son públicas (/api/auth/**) y cuáles requieren autenticación. Configura el modo stateless.
WebConfig	Configura CORS para permitir peticiones desde http://localhost:4200.

5. Webservices — Endpoints REST de la API

Todos los endpoints están bajo la base http://localhost:8080. Los que requieren autenticación deben incluir el header: Authorization: Bearer {token}

5.1 Autenticación — /api/auth

Método	Ruta	Descripción	Parámetros / Body
POST	/api/auth/login	Inicia sesión y retorna JWT	Body JSON: { email, password }
POST	/api/auth/register	Registra un nuevo usuario	Body JSON: objeto Usuario completo

5.2 Solicitudes — /api/solicitudes

Método	Ruta	Descripción	Parámetros / Body
POST	/crear/{estudianteId}	Crea solicitud con ID de estudiante	Path: estudianteId. Body: Solicitud
POST	/crear-por-usuario/{usuarioId}	Crea solicitud con ID del usuario del JWT	Path: usuarioId. Body: Solicitud
GET	/usuario/{usuarioId}	Lista solicitudes del usuario logueado	Path: usuarioId
GET	/estudiante/{estudianteId}	Lista solicitudes de un estudiante	Path: estudianteId
GET	/	Lista todas las solicitudes (admin)	—
GET	/{id}	Obtiene una solicitud por ID	Path: id
POST	/enviar/{id}	Cambia estado a ENVIADA	Path: id
POST	/aprobar/{id}	Aprueba la solicitud (admin)	Path: id
POST	/rechazar/{id}	Rechaza la solicitud (admin)	Path: id

5.3 Anteproyectos — /api/anteproyectos

Método	Ruta	Descripción	Parámetros / Body
POST	/enviar/{solicitudId}	Estudiante sube el PDF (multipart)	Path: solicitudId. Form: archivo (PDF)

GET	/solicitud/{solicitudId}	Consulta el anteproyecto de una solicitud	Path: solicitudId
GET	/ver/{solicitudId}	Visualiza/descarga el PDF en el navegador	Path: solicitudId → devuelve PDF inline
POST	/aprobar/{id}	Admin aprueba el anteproyecto	Path: id. Param: observaciones
POST	/rechazar/{id}	Admin rechaza el anteproyecto	Path: id. Param: observaciones

5.4 Cronograma — /api/cronogramas

Método	Ruta	Descripción	Parámetros / Body
POST	/crear	Programa fecha, hora y sala para una solicitud	Params: solicitudId, salaId, fecha, hora
GET	/	Lista todos los cronogramas (admin)	—
GET	/estudiante/{estudianteId}	Cronogramas de un estudiante	Path: estudianteId
GET	/usuario/{usuarioId}	Cronogramas según usuarioId del JWT	Path: usuarioId
GET	/solicitud/{solicitudId}	Cronograma específico de una solicitud	Path: solicitudId
DELETE	/{id}	Elimina un cronograma programado	Path: id

5.5 Jurados y Tutores — /api/jurados

Método	Ruta	Descripción	Parámetros / Body
POST	/asignar	Asigna un docente como jurado manualmente	Params: solicitudId, docenteId, rol (PRESIDENTE/VOCAL_1/VOCAL_2)
POST	/asignar-automatico/{solicitudId}	Asigna automáticamente los 3 jurados	Path: solicitudId
GET	/solicitud/{solicitudId}	Lista jurados de una solicitud	Path: solicitudId

GET	/	Lista todos los jurados	—
DELETE	/{juradoId}	Elimina un jurado asignado	Path: juradoId
GET	/sugerencias/{solicitudId}	Sugiere docentes disponibles	Path: solicitudId. Param: cantidad (default 5)
POST	/tutor/asignar	Asigna un tutor a una solicitud	Params: solicitudId, docenteId
GET	/tutor/solicitud/{solicitudId}	Obtiene el tutor activo de una solicitud	Path: solicitudId
DELETE	/tutor/{tutorId}	Elimina la asignación de tutor	Path: tutorId
GET	/docente/{docenteId}	Lista asignaciones de jurado de un docente	Path: docenteId
GET	/tutor/docente/{docenteId}	Lista tutorías de un docente	Path: docenteId

5.6 Evaluaciones — /api/evaluaciones

Método	Ruta	Descripción	Parámetros / Body
POST	/evaluar-ponderado	Registra evaluación con ponderación 60/40 configurable	Params: solicitudId, rubricaId, notaInstructor, notaJurado, observaciones, pesoInstructor (def 60), pesoJurado (def 40)
POST	/evaluar	Registro legado con nota final directa	Params: solicitudId, rubricaId, notaFinal, observaciones
GET	/	Lista todas las evaluaciones	—
GET	/estudiante/{estudianteId}	Evaluaciones de un estudiante	Path: estudianteId
GET	/usuario/{usuarioId}	Evaluaciones por usuarioId del JWT	Path: usuarioId
GET	/solicitud/{solicitudId}	Evaluación de una solicitud	Path: solicitudId

5.7 Actas — /api/actas

Método	Ruta	Descripción	Parámetros / Body
POST	/generar/{solicitudId}	Genera el acta y su PDF	Path: solicitudId

POST	/firmar/{actaId}	Firma el acta por rol específico	Path: actaId. Param: rol (PRESIDENTE/VOCAL_1/VOCAL_2/TUTOR)
GET	/descargar/{actaId}	Descarga el PDF del acta (attachment)	Path: actaId
GET	/ver/{actaId}	Visualiza el PDF en el navegador (inline)	Path: actaId
GET	/	Lista todas las actas	—
GET	/solicitud/{solicitudId}	Acta de una solicitud específica	Path: solicitudId

5.8 Notificaciones — /api/notificaciones

Método	Ruta	Descripción	Parámetros / Body
POST	/crear	Crea una notificación para un usuario	Params: usuarioId, mensaje
GET	/	Lista todas las notificaciones	—
GET	/usuario/{usuarioId}	Notificaciones de un usuario	Path: usuarioId
GET	/usuario/{usuarioId}/no-leidas	Cuenta notificaciones no leídas	Path: usuarioId → { total: N }
PATCH	/{id}/marcar-leida	Marca una notificación como leída	Path: id
PATCH	/usuario/{usuarioId}/marcar-todas-leidas	Marca todas las notificaciones como leídas	Path: usuarioId

5.9 Docentes — /api/docentes

Método	Ruta	Descripción	Parámetros / Body
GET	/	Lista todos los docentes	—

GET	/disponibles	Lista docentes disponibles para asignación	—
GET	/{id}	Obtiene un docente por ID	Path: id
GET	/usuario/{usuarioId}	Obtiene el perfil docente por usuarioId	Path: usuarioId

5.10 Salas — /api/salas

Método	Ruta	Descripción	Parámetros / Body
GET	/	Lista todas las salas	—
POST	/	Crea una nueva sala	Body JSON: { codigo, nombre, capacidad, disponible }
DELETE	/{id}	Elimina una sala	Path: id

6. Frontend — Angular 17

6.1 Tecnologías

Framework	Angular 17 (standalone components)
Lenguaje	TypeScript
Estilos	CSS propio por componente
Comunicación HTTP	HttpClient con interceptor JWT
Routing	Angular Router con guards de autenticación
Puerto de desarrollo	4200

6.2 Módulos y Rutas de la Aplicación

El archivo app.routes.ts define la navegación completa del sistema:

/login	Pantalla de inicio de sesión (pública)
/dashboard	Contenedor principal (requiere auth)
/dashboard/solicitudes/registrar	Formulario para registrar nueva solicitud
/dashboard/solicitudes/mis-tramites	Lista de solicitudes del estudiante logueado
/dashboard/solicitudes/cargar-pdf/:id	Subida del PDF de anteproyecto para una solicitud
/dashboard/notas	Vista de notas/evaluaciones del estudiante
/dashboard/horario	Cronograma/horario de sustentación del estudiante
/dashboard/notificaciones	Centro de notificaciones del usuario
/dashboard/admin/revisar-solicitudes	Panel admin para aprobar o rechazar solicitudes
/dashboard/admin/cronograma/:id	Formulario para programar fecha/sala de una solicitud
/dashboard/admin/evaluar/:id	Formulario de evaluación con rubrica y ponderación
/dashboard/docente/anteproyecto/:id	Vista del PDF del anteproyecto para el docente

6.3 Mecanismo de Autenticación en el Frontend

El sistema de autenticación del frontend opera de la siguiente manera:

- Al hacer login exitoso, el AuthService guarda en localStorage el token JWT (presus_token), el id del usuario (user_id), su rol (user_role) y su nombre (user_name).
- El AuthInterceptor (authInterceptor) se registra globalmente en app.config.ts. En cada petición HTTP saliente, lee el token de localStorage y agrega el header Authorization: Bearer {token} de forma automática.

- El AuthGuard (authGuard) protege todas las rutas bajo /dashboard. Si no hay token, redirige automáticamente a /login.
- Al cerrar sesión, logout() limpia todas las claves del localStorage y navega a /login.

6.4 Servicios Angular — Consumo de la API

Cada servicio Angular se corresponde directamente con un grupo de endpoints del backend:

AuthService	Maneja login, logout, y almacenamiento del token. Apunta a /api/auth
SolicitudService	CRUD de solicitudes: crear, listar, enviar, aprobar, rechazar. Apunta a /api/solicitudes
AnteproyectoService	Subida de PDF y consulta de anteproyectos. Apunta a /api/anteproyectos
CronogramaService	Programación y consulta de cronogramas. Apunta a /api/cronogramas
EvaluacionService	Registro y consulta de evaluaciones con ponderación. Apunta a /api/evaluaciones
RubricaService	Consulta de rúbricas disponibles. Apunta a /api/rubricas
NotificacionService	Lectura y marcado de notificaciones. Apunta a /api/notificaciones
SalaService	Consulta de salas disponibles. Apunta a /api/salas

6.5 Componentes Principales

Login (components/auth/login)

Formulario de ingreso con email y contraseña. Al recibir respuesta exitosa del backend, almacena el token y redirige al dashboard según el rol del usuario.

Dashboard (components/dashboard)

Componente contenedor que muestra el menú de navegación lateral adaptado al rol del usuario. Consulta el nombre del usuario desde localStorage y muestra el contador de notificaciones no leídas.

RegistrarSolicitud (components/solicitudes/registrar-solicitudes)

Formulario para que el estudiante registre una nueva solicitud indicando el título del tema, modalidad y observaciones. Usa SolicitudService.registrarSolicitud() con el usuarioId del JWT.

ListarSolicitudes (components/solicitudes/listar-solicitudes)

Tabla que lista las solicitudes del estudiante logueado. Muestra estado, fechas y permite navegar a la carga del PDF si la solicitud fue aprobada.

CargarAnteproyecto (components/solicitudes/cargar-anteproyecto)

Permite al estudiante seleccionar un archivo PDF y subirlo al backend mediante multipart/form-data. Usa AnteproyectoService conectando con POST /api/anteproyectos/enviar/{solicitudId}.

RevisarSolicitudes (components/admin/revisar-solicitudes)

Panel administrativo que lista todas las solicitudes. Permite aprobar, rechazar, ver el anteproyecto y navegar a las vistas de programar cronograma o evaluar.

ProgramarCronograma (components/admin/cronograma)

Formulario donde el admin selecciona una sala disponible, fecha y hora para la sustentación. Llama a POST /api/cronogramas/crear con los parámetros correspondientes.

EvaluarSolicitud (components/admin/evaluacion)

Formulario de evaluación con campos para nota del instructor, nota del jurado, ponderaciones configurables y observaciones. Calcula la nota final previsualizando el resultado antes de enviarlo al backend via POST /api/evaluaciones/evaluar-ponderado.

VerAnteproyecto (components/docente/ver-anteproyecto)

Componente para docentes que muestra el PDF del anteproyecto embebido en el navegador, usando la URL GET /api/anteproyectos/ver/{solicitudId}.

MisNotas (components/notas)

Vista del estudiante que consulta GET /api/evaluaciones/usuario/{usuarioId} y muestra la nota instructor, nota jurado, nota final y resultado (APROBADO/REPROBADO).

MiHorario (components/horario)

Vista del estudiante que consulta GET /api/cronogramas/usuario/{usuarioId} y muestra la fecha, hora, sala y duración de su sustentación programada.

Notificaciones (components/notificaciones)

Lista las notificaciones del usuario, permitiendo marcar como leídas individual o masivamente. Usa NotificacionService conectando con los endpoints PATCH del backend.

7. Flujo Completo del Proceso de Pre-Sustentación

A continuación se describe el proceso completo paso a paso, indicando qué actor lo ejecuta y qué endpoint de la API se consume:

Paso 1	Estudiante inicia sesión → POST /api/auth/login → Recibe JWT con su rol e ID
Paso 2	Estudiante registra su solicitud → POST /api/solicitudes/crear-por-usuario/{usuarioId}
Paso 3	Estudiante envía la solicitud para revisión → POST /api/solicitudes/enviar/{id}
Paso 4	Admin revisa y aprueba o rechaza → POST /api/solicitudes/aprobar/{id} o /rechazar/{id}
Paso 5	Estudiante carga el PDF del anteproyecto → POST /api/anteproyectos/enviar/{solicitudId}
Paso 6	Docente revisa el anteproyecto → GET /api/anteproyectos/ver/{solicitudId}
Paso 7	Admin aprueba o rechaza el anteproyecto → POST /api/anteproyectos/aprobar/{id} o /rechazar/{id}
Paso 8	Admin asigna jurados automáticamente → POST /api/jurados/asignar-automatico/{solicitudId}
Paso 9	Admin asigna tutor → POST /api/jurados/tutor/asignar
Paso 10	Admin programa la fecha y sala → POST /api/cronogramas/crear
Paso 11	Estudiante consulta su horario → GET /api/cronogramas/usuario/{usuarioId}
Paso 12	Admin registra la evaluación con ponderación → POST /api/evaluaciones/evaluar-ponderado
Paso 13	Admin genera el acta → POST /api/actas/generar/{solicitudId}
Paso 14	Cada actor firma el acta por su rol → POST /api/actas/firmar/{actaId}?rol=PRESIDENTE|VOCAL_1|VOCAL_2|TUTOR
Paso 15	Estudiante descarga el acta final → GET /api/actas/descargar/{actaId}
Paso 16	Estudiante consulta sus notas finales → GET /api/evaluaciones/usuario/{usuarioId}

8. Cómo Ejecutar el Sistema

8.1 Prerrequisitos

- Java 17 o superior instalado
- Maven 3.8+ instalado
- Node.js 18+ y Angular CLI instalados (npm install -g @angular/cli)
- PostgreSQL instalado y corriendo

8.2 Configurar la Base de Datos

- Crear una base de datos en PostgreSQL llamada presusDb
- Ejecutar el script datos-iniciales.sql incluido en el proyecto backend para cargar datos de prueba
- Opcionalmente configurar las variables de entorno DB_URL, DB_USERNAME y DB_PASSWORD

8.3 Ejecutar el Backend

- Abrir terminal en la carpeta backend-mod/
- Ejecutar: mvn spring-boot:run
- El servidor arranca en http://localhost:8080
- Spring Boot creará/actualizará las tablas automáticamente (ddl-auto=update)

8.4 Ejecutar el Frontend

- Abrir terminal en la carpeta frontend-output/
- Ejecutar: npm install
- Ejecutar: ng serve
- La aplicación estará disponible en http://localhost:4200